

TCP01 · Streams e Serialização de Objetos

Respostas, tabelas preenchidas e registo das execuções

Todos os resultados indicados neste documento foram obtidos a correr o código entregue (dois projetos, tcp01-cliente e tcp01-servidor, pacote tcp01, porto 7896, servidor arrancado sempre primeiro). As saídas de consola aparecem tal como foram impressas. O código está organizado por etapa (etapa-4.1, etapa-4.2, etapa-4.3); a versão final é a da etapa 4.3.

1 · Diagnóstico inicial (2.2)

(a) O que o TCP garante a quem envia dados, e o que o UDP não garante.

O TCP garante que os bytes enviados chegam ao outro lado sem corrupção (checksums), pela mesma ordem (numeração dos segmentos), sem duplicados e sem falhas (confirmações e retransmissão após timeout), dentro de uma ligação estabelecida previamente. O UDP envia datagramas independentes: podem perder-se, chegar fora de ordem ou duplicados, e a aplicação não é avisada. O TCP **não** garante que o recetor saiba interpretar esses bytes — isso é responsabilidade da aplicação.

(b) O que acontece a um programa que chama `accept()` ou `readUTF()` quando ainda não há nada para receber.

A thread que faz a chamada fica bloqueada: `accept()` só devolve quando um cliente se ligar, `readUTF()` só devolve quando tiver recebido uma string completa (2 bytes de comprimento + o conteúdo). Se a ligação for fechada antes disso, `readUTF()` lança `EOFException`.

(c) Por que razão um objeto tem de ser convertido em bytes, e por que razão o outro lado precisa da classe.

Uma stream TCP só transporta bytes; um objeto em memória é um conjunto de valores e referências (endereços) que não têm significado noutra máquina. Serializar escreve os valores dos atributos e a descrição da classe (nome completo e `serialVersionUID`). Os bytes não contêm o código da classe, por isso quem lê precisa da mesma classe localmente para criar o objeto e voltar a preencher os seus atributos.

2 · Etapa 4.1 — Trocar texto

Linhas que bloqueiam (numeração dos ficheiros entregues em etapa-4.1):

Ficheiro : linha	Instrução	O que está à espera
TCPServer.java : 12	<code>listenSocket.accept()</code>	Que um cliente estabeleça uma ligação ao porto 7896.
Connection.java : 25	<code>in.readUTF()</code>	Que o cliente envie uma string completa (ou feche a ligação → <code>EOFException</code>).
TCPClient.java : 11	<code>new Socket("localhost", 7896)</code>	Que o handshake TCP termine. Não fica à espera indefinidamente: se ninguém escuta no porto, falha logo.
TCPClient.java : 15	<code>in.readUTF()</code>	A resposta do servidor (ou o fecho da ligação → <code>EOFException</code>).
(nota) writeUTF	<code>out.writeUTF(...)</code>	Normalmente não bloqueia: copia para o buffer de envio do SO. Só bloquearia se esse buffer estivesse cheio porque o outro lado não lê.

Execução normal (servidor e depois cliente):

[cliente] Received: mensagem em UTF

Cliente com o servidor parado: falhou na linha 11 do `TCPClient` (`new Socket(...)`), com `java.net.ConnectException: Connection refused` — subclasse de `IOException`, por isso é apanhada no catch (`IOException` e) e impressa como:

IO: Connection refused

3 · Etapa 4.2 — Enviar um objeto

Onde tem de existir a classe Person?

Nos **dois** projetos, no mesmo pacote tcp01. O cliente precisa dela para criar o objeto e serializá-lo; o servidor precisa dela para o reconstruir (a stream só traz o nome da classe e os valores dos atributos, não o código).

Alterações feitas:

- Person implements Serializable, com private static final long serialVersionUID = 1L, construtor Person(String name, int year) e getters getName()/getYear().
- Cliente: a saída passa a ser um ObjectOutputStream (writeObject(person) + flush()); a entrada continua DataInputStream (readUTF()).
- Servidor (Connection): a entrada passa a ObjectInputStream; readObject() seguido de instanceof + cast para Person; resposta com writeUTF(p.getName()); catch (ClassNotFoundException e) obrigatório.

```
[cliente] Received: Maria Silva
[servidor] Recebido: Maria Silva (2001)
```

4 · Etapa 4.3 — Objeto com dependências

Que requisitos tem o Place de cumprir, e onde tem de existir?

Tem de implementar Serializable (todas as classes alcançáveis a partir da Person têm de o ser) e tem de existir nos dois projetos, no pacote tcp01, com serialVersionUID compatível. Os seus atributos (String) já são serializáveis.

O cliente só chama writeObject(person); o servidor devolve p.getPlace().getLocality():

```
[cliente] Received: Viseu
[servidor] Recebido: Maria Silva (2001), 3500-001 Viseu
```

A localidade chegou sem nenhuma escrita explícita do Place: é a prova de que foi serializado automaticamente com a Person.

Variantes de versão (lógica do programa inalterada):

Variante	Alteração	Resultado
Continua a funcionar	Acrescentado isAdult(int) à Person nos dois projetos (serialVersionUID = 1L mantido).	Received: Viseu. Métodos não fazem parte do estado serializado e o UID é o mesmo.
Continua a funcionar (extra)	Acrescentado o atributo phoneNumber só à Person do servidor, sem mexer no UID.	Received: Viseu. O campo que não veio na stream fica com o valor por omissão (null).
Deixa de funcionar	serialVersionUID = 2L na Person do servidor (1L no cliente).	Servidor , em readObject(): InvalidClassException: tcp01.Person; local class incompatible: stream classdesc serialVersionUID = 1, local class serialVersionUID = 2. Cliente: EOF: null no readUTF().
Deixa de funcionar (extra)	UID não declarado nos dois lados e um método novo só no servidor.	Servidor: InvalidClassException com UIDs calculados diferentes (1681929146346088193 vs -2969063439951136928). Mostra porque se declara o UID explicitamente.

5 · Etapa 4.4 — Verificação

Tabela de falhas (cada falha introduzida isoladamente sobre o código da etapa 4.3, reposta a seguir):

Falha introduzida	Lado onde surgiu	Exceção obtida	Momento	O que a exceção permitiu (ou não) concluir
Arrancar o cliente sem o servidor	Cliente	<code>java.net.ConnectException: Connection refused (impressa como «IO: Connection refused»)</code>	Ligação (new Socket)	Que não há ninguém à escuta em localhost:7896. Não diz porquê (servidor parado, porto errado, firewall). Nenhum byte chegou a ser enviado.
Retirar implements Serializable do Place	Cliente (origem) e servidor (con sequência)	Cliente: <code>NotSerializableException: tcp01.Place</code> Servidor: <code>WriteAbortedException: writing aborted; java.io.NotSerializableException: tcp01.Place</code>	Cliente: escrita (<code>writeObject</code>) Servidor: leitura (<code>readObject</code>)	Que a falha é do Place e não da Person: o grafo inteiro tem de ser serializável. A exceção do servidor só traz a causa porque o cliente conseguiu escrever o marcador de abortado na stream; o problema não é do servidor.
serialVersionUID da Person diferente nos dois lados	Servidor (quem lê)	<code>InvalidClassException: tcp01.Person; local class incompatible: stream classdesc serialVersionUID = 1, local class serialVersionUID = 2</code> Cliente: <code>EOFException (EOF: null)</code>	Servidor: leitura (<code>readObject</code>) Cliente: leitura da resposta	Os bytes chegaram intactos, mas as versões da classe são incompatíveis. O servidor sabe exatamente porquê; o cliente só vê que a ligação foi fechada sem resposta.
Person num pacote diferente no servidor (servidor.Person)	Servidor	<code>ClassNotFoundException: tcp01.Person</code> Cliente: <code>EOFException (EOF: null)</code>	Servidor: leitura (<code>readObject</code>) Cliente: leitura da resposta	Que a identidade de uma classe é o nome completo, pacote incluído: o código igual em servidor.Person não serve. O cliente, mais uma vez, não fica a saber a causa.

Tabela de construções:

Construção	Onde a usou	O que ficou a ser garantido	O que continua a não ser garantido
ServerSocket / accept()	TCPServer.main, linhas 10-12	O servidor escuta no porto 7896 e obtém um Socket próprio para cada cliente que se liga.	Que o cliente envie alguma coisa, que seja quem diz ser, ou que o número de ligações aceites seja sustentável.
Connection extends Thread	Connection (construtor chama start(); trabalho em run())	Cada cliente é atendido na sua thread; um cliente lento não impede o accept() de outros (demonstrado).	Limite de recursos (uma thread por ligação); nenhum timeout — um cliente parado ocupa uma thread indefinidamente.
implements Serializable	Person e Place, nos dois projetos	Que os objetos podem ser convertidos em bytes e reconstruídos pela JVM.	Que o outro lado tenha a mesma classe; controlo do que é enviado (vai todo o estado, incluindo dados sensíveis); validade dos valores.
ObjectOutputStream / ObjectOutputStream	Cliente: <code>writeObject(person)</code> ; servidor: <code>readObject()</code>	Envio do objeto completo, com o grafo de dependências, numa única chamada.	Tipo do objeto recebido (é Object → cast/instanceof); interoperabilidade com outras linguagens; segurança do que é desserializado.
serialVersionUID	Person e Place: = 1L	Compatibilidade controlada à mão: métodos/atributos novos não quebram a leitura; versões incompatíveis são detetadas (InvalidClassException).	Que as cópias da classe estejam sincronizadas; que mudanças compatíveis segundo o UID tenham o mesmo significado (ex.: campo novo fica null).
Referência para Place	Person.place; servidor usa <code>getPlace().getLocality()</code>	O Place viaja automaticamente com a Person (Received: Viseu).	Que o Place seja serializável (senão falha a escrita inteira); controlo do volume — tudo o que é alcançável é enviado; o place pode ser null.

6 · Reflexão crítica (4.5)

1. Que problemas encontrados não eram de transmissão? Quem tinha de os resolver?

Classe não serializável (Place), versões incompatíveis (serialVersionUID), classe noutra pacote, e o tipo devolvido por `readObject()` (Object, exige cast). Em todos os casos os bytes chegaram intactos e por ordem; o problema era de *interpretação*. Quem os resolve é a aplicação — o programador, ao garantir que as classes existem e são iguais nos dois lados, que todo o grafo é serializável e que o recetor verifica o que recebe.

2. Num sistema real com vários clientes, o que acontece quando o servidor muda a Person? Papel do serialVersionUID?

Os clientes antigos continuam a enviar objetos da versão antiga. Se a mudança for compatível (método novo, atributo acrescentado) e o UID se mantiver, o servidor aceita — os campos novos ficam com valores por omissão. Se a mudança for incompatível e o UID for alterado, o servidor passa a rejeitar os clientes antigos com `InvalidClassException` e todos têm de ser atualizados. O UID é o contrato de versão: declarado à mão, permite decidir conscientemente quando se quebra a compatibilidade; se não for declarado, qualquer recompilação com alterações pode quebrá-la sem aviso (variante extra da etapa 4.3).

3. Riscos de enviar todo o grafo alcançável?

Volume: uma referência aparentemente inocente pode arrastar listas, caches ou estruturas grandes, e o custo não se vê no código do cliente (só se escreve `writeObject(person)`). Informação enviada sem querer: qualquer atributo, mesmo privado (uma palavra-passe, dados pessoais), é transmitido. Fragilidade: basta um objeto do grafo não ser serializável para toda a escrita falhar. Com os mecanismos desta ficha, a única defesa é desenhar as classes enviadas para conterem apenas o que deve viajar.

4. Uma thread por ligação vs. um cliente de cada vez?

Ganha-se concorrência: um cliente lento ou parado não impede o atendimento dos outros (demonstrado: com `start()` o segundo cliente recebeu resposta enquanto outro estava ligado sem enviar nada; com `run()` o segundo cliente ficou bloqueado até ser cortado por `timeout`). Passa a gastar-se: criação e destruição de uma thread por ligação, memória (pilha de cada thread), comutação de contexto e o risco de esgotar recursos com muitos clientes. Atender um de cada vez é mais barato e simples, mas o tempo de resposta de cada cliente depende de todos os anteriores.

5. Quando preferir datagramas a streams? (exemplo diferente do video streaming)

Quando a informação perde valor se chegar atrasada ou quando cada mensagem é pequena e independente, e o custo de estabelecer ligação e retransmitir é maior do que o de perder um pacote: jogos multijogador em tempo real (a posição de um jogador de há 200 ms já não interessa — é melhor usar a seguinte), voz sobre IP, consultas DNS (pedido/resposta de um só pacote; se falhar, repete-se) ou telemetria de sensores enviada periodicamente.

7 · Perguntas de verificação (critérios de aceitação)

CA1 · Modelo de comunicação TCP

Percorso de uma mensagem. `writeUTF` escreve 2 bytes com o comprimento e os bytes da string (UTF-8 modificado) para o buffer de envio do socket. O SO divide os bytes em segmentos TCP numerados, com checksum; o SO do servidor verifica o checksum, confirma (ACK), reordena e pede retransmissão do que se perder, e entrega a sequência à stream de entrada. `readUTF` lê os 2 bytes do comprimento e depois exatamente esse número de bytes. As operações que bloqueiam estão na tabela da etapa 4.1 (e, na versão final, também a criação do `ObjectInputStream` — `Connection.java` linha 17 — e `readObject()` na linha 28).

Cliente antes do servidor?

Falha na criação do socket (`TCPClient.java` linha 11) com `ConnectException: Connection refused`. `s` fica null, por isso o `finally` não tem nada para fechar.

O TCP garante que o servidor percebeu a mensagem?

Não. Exemplos do trabalho: com `serialVersionUID` diferente e com a `Person` noutro pacote, todos os bytes chegaram corretamente e o pedido falhou no servidor (`InvalidClassException` / `ClassNotFoundException`).

ServerSocket + Socket no servidor, só Socket no cliente? Quantos sockets com dois clientes?

O `ServerSocket` não troca dados: só escuta e aceita pedidos de ligação. Cada `accept()` devolve um `Socket` para comunicar com aquele cliente. O cliente só precisa de iniciar uma ligação, logo um `Socket`. Com dois clientes ligados: 3 sockets no servidor (1 `ServerSocket` + 2 `Socket`).

O que vê o cliente se o servidor fechar sem responder?

`readUTF()` lança `EOFException` — observado como «EOF: null» nas falhas de versão e de pacote. (Se a ligação fosse cortada abruptamente, poderia ser `SocketException: Connection reset`.)

CA2 · Concorrência no servidor

Onde começa a nova thread? E se chamasse run() em vez de start()?

A thread nova é criada quando o construtor chama `start()` (o construtor em si corre na thread principal); o código que ela executa é o `run()`. Chamar `run()` diretamente executa-o na thread principal, como uma chamada normal: o servidor fica sequencial.

Sem Connection, o que acontece a um segundo cliente enquanto o primeiro não envia nada? Como demonstrar?

O SO aceita a ligação do segundo cliente (o handshake completa-se na fila do `ServerSocket`), o cliente envia, mas fica bloqueado no `readUTF()` porque a thread principal está presa no `readUTF()` do primeiro. Demonstração feita: um primeiro cliente liga-se e fica 6 s sem enviar; um segundo `TCPClient` com limite de 4 s. Com `start()`: recebe «mensagem em UTF» de imediato. Com `run()`: termina por timeout sem resposta, e só depois de o primeiro sair um novo cliente é atendido.

Quantas threads com três clientes? E depois?

4 threads da aplicação: a principal (no `accept()`) + 3 `Connection`. Depois de terminarem, fica só a principal (sem contar as threads internas da JVM, como o garbage collector).

Porque se fecha o socket no finally do run()?

Só a `Connection` sabe quando acabou de usar o socket. Se a thread principal o fechasse logo a seguir a criar a `Connection`, a outra thread falharia a ler/escrever; e se esperasse pelo fim, perdia-se a concorrência. O `finally` garante o fecho também quando há exceções (`EOF`, `InvalidClass`, ...).

CA3 · Serialização de objetos

Person sem Serializable — em que lado surge o erro?

No lado que escreve: `writeObject` lança `NotSerializableException` no cliente. O servidor recebe o marcador de escrita abortada e vê `WriteAbortedException` (comportamento observado com o `Place`, que é o mesmo mecanismo).

Porque é preciso o cast? E se chegar outra classe?

`readObject()` pode devolver qualquer objeto, o compilador só sabe que é `Object`. O `cast` diz ao compilador qual é o tipo esperado. Se chegar outra classe, um `cast` direto lançaria `ClassCastException`; por isso o servidor testa com `instanceof` e responde com uma mensagem de erro. Se a classe não existir no servidor, a falha é antes, no próprio `readObject()` (`ClassNotFoundException`).

A Person tem de ser exatamente igual nos dois projetos? E o pacote?

Não tem de ser igual em tudo: tem de ter o mesmo nome completo, o mesmo `serialVersionUID` e atributos compatíveis. Demonstrado: método novo nos dois lados e atributo extra só no servidor continuam a funcionar. O pacote tem de ser o mesmo: com servidor `Person` o servidor lança `ClassNotFoundException: tcp01.Person`.

Entrada de objetos e saída de dados no servidor — porque funciona? E se devolvesse um objeto?

Os dois sentidos de uma ligação TCP são streams independentes. O que importa é que cada sentido use o mesmo formato nas duas pontas: cliente `ObjectOutputStream` → servidor `ObjectInputStream`; servidor `DataOutputStream` → cliente `DataInputStream`. Se o servidor devolvesse um objeto, passaria a `ObjectOutputStream` e o cliente a `ObjectInputStream`; como ambos os sentidos teriam objetos, cada lado teria de criar primeiro o `ObjectOutputStream` (e fazer `flush()`), senão os dois ficariam bloqueados à espera do cabeçalho um do outro.

CA4 · Dependências, versões e interpretação de erros

Como é que o Place chegou ao servidor? O que tem de ter?

`writeObject` percorre todas as referências a partir da `Person` e serializa o grafo completo. O `Place` tem de implementar `Serializable` e existir no servidor com o mesmo nome completo e UID compatível.

Place sem Serializable: o que vê o servidor? A exceção diz a causa verdadeira?

Vê `WriteAbortedException: writing aborted; java.io.NotSerializableException: tcp01.Place`. Neste caso diz, porque a JVM do cliente escreve na stream a indicação de que a escrita foi abortada. Mas a exceção surge no servidor e o erro é do cliente; se o cliente terminasse sem conseguir escrever esse marcador, o servidor veria apenas fim de stream, sem causa.

serialVersionUID diferente: quem rejeitou e quando? E atributo novo sem mexer no UID?

Rejeitou o servidor (o lado que lê), no `readObject()`, com `InvalidClassException`. O cliente escreveu sem erro e só viu EOF ao esperar a resposta. Atributo novo com o mesmo UID: funciona — o campo em falta fica com o valor por omissão (null/0) e campos que o recetor não conhece são ignorados.

EOFException no servidor: erro do servidor, do cliente ou da rede?

Por si só indica que o outro lado fechou a ligação antes de enviar o que se esperava — tipicamente o cliente (no teste de concorrência, o cliente que se ligou e saiu sem enviar produziu «EOF: null» no servidor). Distinguir: **cliente** — fecho ordenado (EOF) e o log do cliente mostra o motivo (exceção na escrita, fim do programa); **rede** — normalmente `SocketException` (connection reset, timeout) ou leitura bloqueada indefinidamente, não EOF limpo; **servidor** — o servidor lança uma exceção sobre os dados recebidos (`InvalidClass`, `ClassNotFound`, `ClassCast`), e é então o cliente que vê o EOF.

CA5 · Limites da solução

Limitação escolhida: o servidor confia em qualquer `Person` que receba. A desserialização reconstrói o objeto *sem chamar o construtor*, por isso nenhuma verificação no construtor protege o servidor. Faltaria validar explicitamente os dados no servidor (em `Connection.run()`, depois do cast) e responder com erro quando forem implausíveis; e, de forma mais geral, um formato de troca independente das classes Java — que fica fora desta ficha.

Atributo que não deve ser enviado (palavra-passe)?

Com os mecanismos desta ficha, não: todo o estado do objeto é serializado. A única forma é desenhar a classe para não conter esse dado (mantê-lo noutro objeto que não seja referenciado pela `Person`). A serialização de base não resolve este problema; a resposta fica para mais tarde.

Um servidor noutra linguagem consegue ler os objetos?

Na prática, não: o formato é específico do Java e a reconstrução exige as próprias classes Java. A serialização nativa só serve para comunicação Java↔Java com as mesmas classes nos dois lados.

Dez mil clientes em simultâneo?

Dez mil threads, cada uma com a sua pilha e custo de escalonamento: consumo de memória elevado, muita comutação de contexto e, a certa altura, falha na criação de threads (`OutOfMemoryError: unable to create native thread`) ou esgotamento de descritores de ficheiro. Com os mecanismos desta ficha não há solução; fica para mais tarde.

O year é plausível?

Não é verificado. Teria de ser verificado no servidor, depois de `readObject()` — nunca se pode confiar que o cliente validou, porque qualquer programa pode enviar uma `Person` com valores arbitrários.

Anexo · Código final (etapa 4.3)

tcp01-cliente/src/tcp01/TCPClient.java

```
package tcp01;

import java.io.*;
import java.net.*;

public class TCPClient {
    public static void main(String[] args) {
        Socket s = null;
        try {
            int serverPort = 7896; // porto do servidor
            s = new Socket("localhost", serverPort); // BLOQUEIA até a ligação ser aceite (ou falha)
            ObjectOutputStream out = new ObjectOutputStream(s.getOutputStream()); // escreve o cabeçalho
            DataInputStream in = new DataInputStream(s.getInputStream());

            Place place = new Place("3500-001", "Viseu");
            Person person = new Person("Maria Silva", place, 2001);
            out.writeObject(person); // envia a Person (e o Place com ela)
            out.flush();

            String data = in.readUTF(); // BLOQUEIA à espera da resposta
            System.out.println("Received: " + data);
        } catch (UnknownHostException e) {
            System.out.println("Sock: " + e.getMessage());
        } catch (EOFException e) {
            System.out.println("EOF: " + e.getMessage());
        } catch (IOException e) {
            System.out.println("IO: " + e);
        } finally {
            if (s != null) {
                try {
                    s.close();
                } catch (IOException e) {
                    System.out.println("close: " + e.getMessage());
                }
            }
        }
    }
}
```

tcp01-servidor/src/tcp01/TCPServer.java

```
package tcp01;

import java.io.*;
import java.net.*;

public class TCPServer {
    public static void main(String[] args) {
        try {
            int serverPort = 7896;
            ServerSocket listenSocket = new ServerSocket(serverPort);
            while (true) {
                Socket clientSocket = listenSocket.accept(); // BLOQUEIA: espera que um cliente se ligue
                Connection c = new Connection(clientSocket); // processa o pedido noutra thread
            }
        } catch (IOException e) {
            System.out.println("Listen: " + e.getMessage());
        }
    }
}
```

tcp01-servidor/src/tcp01/Connection.java

```
package tcp01;

import java.io.*;
import java.net.*;

public class Connection extends Thread {
    ObjectInputStream in;        // entrada: objetos (Person)
    DataOutputStream out;        // saída: texto (localidade)
    Socket clientSocket;

    public Connection(Socket aClientSocket) {
        try {
            clientSocket = aClientSocket;
            // Só o sentido cliente->servidor transporta objetos, por isso não há
            // risco de bloqueio mútuo ao criar o ObjectInputStream.
            // BLOQUEIA: espera pelo cabeçalho que o ObjectOutputStream do cliente escreve.
            in = new ObjectInputStream(clientSocket.getInputStream());
            out = new DataOutputStream(clientSocket.getOutputStream());
            this.start();                                // executa run() numa thread separada
        } catch (IOException e) {
            System.out.println("Connection: " + e.getMessage());
        }
    }

    @Override
    public void run() {
        try {
            Object obj = in.readObject();                // BLOQUEIA: espera um objeto completo
            if (obj instanceof Person) {
                Person p = (Person) obj;                // cast para a classe esperada
                String locality = (p.getPlace() != null) ? p.getPlace().getLocality() : "(sem local)";
                System.out.println("Recebido: " + p);
                out.writeUTF(locality);                  // devolve a localidade como texto
            } else {
                out.writeUTF("Erro: objeto recebido não é uma Person ("
                    + (obj == null ? "null" : obj.getClass().getName()) + ")");
            }
        } catch (ClassNotFoundException e) {
            System.out.println("ClassNotFoundException: " + e.getMessage());
        } catch (EOFException e) {
            System.out.println("EOF: " + e.getMessage());
        } catch (IOException e) {
            System.out.println("IO: " + e);
        } finally {
            try {
                clientSocket.close();
            } catch (IOException e) {
                /* falha ao fechar */
            }
        }
    }
}
```

tcp01-servidor/src/tcp01/Person.java (igual nos dois projetos)

```
package tcp01;

import java.io.Serializable;

public class Person implements Serializable {
    private static final long serialVersionUID = 1L;

    private String name;
    private Place place;
    private int year;

    public Person(String name, Place place, int year) {
        this.name = name;
        this.place = place;
        this.year = year;
    }

    public String getName() { return name; }
    public Place getPlace() { return place; }
    public int getYear() { return year; }

    @Override
    public String toString() {
        return name + " (" + year + "), " + place;
    }
}
```

tcp01-servidor/src/tcp01/Place.java (igual nos dois projetos)

```
package tcp01;

import java.io.Serializable;

public class Place implements Serializable {
    private static final long serialVersionUID = 1L;

    private String postalCode;
    private String locality;

    public Place(String postalCode, String locality) {
        this.postalCode = postalCode;
        this.locality = locality;
    }

    public String getPostalCode() { return postalCode; }
    public String getLocality() { return locality; }

    @Override
    public String toString() {
        return postalCode + " " + locality;
    }
}
```